UNIVERSITY OF JYVÄSKYLÄ

Agent Technologies for Developers

Multi-Agent Visa and Immigration Application Assistant using SPADE

Second Project Report: First Agents Using SPADE

GROUP 95

Syed Mohammad Fahad Zahid
Wajahat Haider
Turna Saha
Uchchwas Saha

1. Introduction

This report covers the second project for Agent Technologies for Developers at the University of Jyväskylä. The goal was to move from conceptual modelling to a working Multi-Agent System (MAS) prototype built with SPADE (Smart Python Agent Development Environment).

The chosen domain is visa and immigration assistance. International applicants face a lot of friction: eligibility rules are not always clear, required documents vary by permit type, and financial requirements are easy to miss. One mistake can delay the whole application.

A multi-agent approach fits well here. The application process breaks naturally into separate concerns: checking eligibility, verifying documents, and coordinating the overall flow. Each agent owns one of those concerns, which mirrors how real immigration offices actually work, with separate staff for eligibility, documents, and case management.

The prototype covers one scenario: a student applying for a Finnish residence permit. The student submits a profile and documents. The system checks eligibility and then verifies whether the documents and available funds are enough. It is a focused prototype, not a production system.

2. Project Requirements and Compliance

> *All four requirements are met. The prototype uses three agents rather than the minimum two, all communicating over XMPP with domain-specific JSON messages.*

Requirement	Status	Implementation Detail
At least two agents implemented	Satisfied	Three agents: OrchestratorAgent, EligibilityAgent, DocumentAgent
Agents communicate via platform messaging	Satisfied	SPADE XMPP messaging with ACL-style performatives and ontology fields
Real domain interactions between agents	Satisfied	Service request, delegated eligibility assessment, document verification
Each agent shows autonomous behaviour	Satisfied	Each agent independently evaluates input and returns a decision
Communication fits domain context	Satisfied	JSON payloads model immigration profile data and document packages

3. System Architecture Overview

The system runs three SPADE agents, each with its own CyclicBehaviour. Agents only talk to each other through XMPP messages; there are no direct function calls between them. This keeps things loosely coupled, so each agent can be developed and tested on its own.

The design follows an orchestrator-specialist pattern. One central agent manages the workflow and hands off tasks to two specialist agents. This lines up with the GAIA role model from the first project phase, where the orchestrator and specialist roles were already kept separate.

A local Prosody server handles XMPP. Each agent gets its own JID on localhost. Messages are JSON strings inside standard SPADE Message objects. The performative and ontology fields on each message make the intent clear without having to parse the body.

The pipeline works like this: the OrchestratorAgent starts with a fixed applicant profile, sends it to the EligibilityAgent, and if the result is positive, forwards the document data to the DocumentAgent. It then takes both results and prints a final recommendation. No human input is needed after the agents start.

4. Implemented Agents

This section covers each agent in detail: what it does, how it communicates, how it makes decisions, and how it relates to the GAIA design from the first project phase.

4.1 OrchestratorAgent

The OrchestratorAgent runs the whole workflow. It starts the process, sends tasks to the specialist agents, collects their responses, and prints the final guidance.

Role and Responsibilities

- Initialise and start the student residence permit application workflow.
- Construct and transmit the applicant profile as a JSON payload to the EligibilityAgent.
- Receive and interpret the eligibility decision returned by the EligibilityAgent.
- Decide autonomously whether the workflow should continue to document verification or terminate.
- Construct and transmit the document data payload to the DocumentAgent if eligibility is confirmed.
- Receive and interpret the document verification result from the DocumentAgent.
- Produce and display the final application recommendation to the console.

Workflow State Machine

The OrchestratorAgent uses a simple state machine inside WorkflowBehaviour. The states are:

State	Description	Next State
START	Workflow initialised; sends eligibility request	WAITING_FOR_ELIGIBILITY
WAITING_FOR_ELIGIBILITY	Awaiting eligibility result from EligibilityAgent	WAITING_FOR_DOCUMENTS or COMPLETED
WAITING_FOR_DOCUMENTS	Awaiting document result from DocumentAgent	COMPLETED
COMPLETED	Final result printed; workflow ends	End

Autonomous Decision Logic

The OrchestratorAgent makes its own decisions at two points:

- After the eligibility result arrives: if the applicant is eligible, the agent moves to document verification. If not, it stops.
- After the document result arrives: if documents are accepted, it reports the application is ready. If there is an issue, it reports exactly what the problem is.

This is genuine autonomous behaviour. The orchestrator is not just a message relay; it reads each response and decides what to do next.

Connection to GAIA Role Model

In the GAIA design, the Orchestrator was the central case manager: coordinates specialists, owns workflow state, produces the final output. This implementation does exactly that.

4.2 EligibilityAgent

The EligibilityAgent checks whether an applicant qualifies for a permit. In this prototype, it handles the Finnish Student Residence Permit.

Role and Responsibilities

- Listen for eligibility assessment requests from the OrchestratorAgent.
- Parse the received JSON applicant profile payload.
- Evaluate the applicant's purpose of visit, destination country, and admission letter status.
- Apply a deterministic rule-based decision to determine eligibility.
- Construct and transmit a structured eligibility result back to the OrchestratorAgent.

Autonomous Decision Rule

The EligibilityAgent runs this rule on its own:

```
IF purpose == 'study'
  AND destination_country == 'Finland'
  AND admission_letter == true
THEN
  eligible = true
  visa_type = 'Student Residence Permit'
  reason = 'Applicant is studying in Finland and has a valid admission letter'
ELSE
  eligible = false
  visa_type = None
  reason = 'Manual Review Required'
```

Response Structure

The EligibilityAgent's response message contains three fields:

- eligible: boolean indicating whether the applicant qualifies.
- visa_type: the recommended permit type if eligible, otherwise null.
- reason: a human-readable explanation of the decision.

Autonomous Behaviour

The EligibilityAgent is autonomous because it evaluates the profile against immigration rules and returns a decision on its own. It never asks the orchestrator for clarification. The result comes entirely from its own rules and the data it received.

Connection to GAIA Role Model

In the GAIA design, the Eligibility Assessment Agent mapped applicant profiles to permit categories using domain knowledge. This agent does that. The rule encodes the eligibility logic for the Finnish Student Residence Permit from the Migrify domain specification.

4.3 DocumentAgent

The DocumentAgent checks two things: whether the submitted documents are complete and whether the available funds meet the minimum threshold.

Role and Responsibilities

- Listen for document verification requests from the OrchestratorAgent.
- Parse the received JSON document payload.
- Check whether all mandatory documents are present in the payload.
- Verify that the declared available funds meet or exceed the minimum threshold.
- Construct and transmit a structured verification result back to the OrchestratorAgent.

Mandatory Document Requirements

Finnish immigration requires four documents for the Student Residence Permit. The DocumentAgent checks for all of them:

Document Field	Description	Required
passport	Valid travel document	Yes
admission_letter	Acceptance letter from Finnish university	Yes
insurance	Proof of health or travel insurance	Yes
bank_statement	Recent bank statement	Yes

Financial Threshold

The minimum required funds are EUR 6,720, based on Finnish immigration subsistence requirements. If the declared available_funds_eur is below that, the agent flags an insufficient funds issue.

Autonomous Decision Rules

```
IF any mandatory document field is missing or false
  THEN result = DOCUMENT_ISSUE
      issue = 'Missing documents: [list of missing fields]'

ELSE IF available_funds_eur < 6720
  THEN result = DOCUMENT_ISSUE
      issue = 'Insufficient funds'
      required_funds_eur = 6720
      available_funds_eur = [declared value]

ELSE result = DOCUMENTS_ACCEPTED
    issue = null
```

Autonomous Behaviour

The DocumentAgent is autonomous: it checks the documents and the funds on its own and picks the right result category without waiting for further instructions. It holds the domain knowledge about Finnish requirements and applies it independently.

Connection to GAIA Role Model

In the GAIA design, the Document Verification Agent was the specialist for document and financial checks before submission. This agent does that. The document list and financial threshold come from real Finnish immigration requirements gathered during the GAIA analysis.

5. Communication Flow and Message Design

Agents only communicate through SPADE XMPP messages. No function calls, no shared memory. Each message is a SPADE Message object with a JSON body and metadata that identifies its type.

5.1 Message Metadata Design

Each SPADE Message carries performative and ontology fields alongside the body. These do two things: they make the message intent readable without parsing the JSON, and they let each agent filter out messages that are not meant for it.

Step	Sender	Receiver	Performative	Ontology
1	OrchestratorAgent	EligibilityAgent	request	immigration-eligibility
2	EligibilityAgent	OrchestratorAgent	inform	immigration-eligibility-result
3	OrchestratorAgent	DocumentAgent	request	document-verification
4	DocumentAgent	OrchestratorAgent	inform	document-verification-result

5.2 Complete Communication Flow

The full message sequence for the student residence permit application:

1. The OrchestratorAgent initialises, transitions to state START, and constructs the applicant profile JSON payload.
2. The OrchestratorAgent sends a request message to the EligibilityAgent's XMPP JID with the performative 'request' and the ontology 'immigration-eligibility'. The message body contains the full applicant profile as a JSON string.
3. The EligibilityAgent's CyclicBehaviour receives the message, parses the JSON payload, applies the eligibility rule, and constructs a result payload containing the eligible flag, visa_type, and reason fields.
4. The EligibilityAgent sends an inform message back to the OrchestratorAgent's JID with the ontology 'immigration-eligibility-result'.
5. The OrchestratorAgent receives the eligibility result and evaluates it autonomously. If the applicant is eligible, it transitions to state WAITING_FOR_DOCUMENTS and proceeds. If not eligible, it transitions to COMPLETED and terminates.
6. The OrchestratorAgent constructs the document payload JSON and sends a request message to the DocumentAgent's JID with the ontology 'document-verification'.
7. The DocumentAgent's CyclicBehaviour receives the message, parses the document payload, checks mandatory documents and available funds, and constructs a verification result.

8. The DocumentAgent sends an inform message back to the OrchestratorAgent with the ontology 'document-verification-result'.
9. The OrchestratorAgent receives the document result, evaluates it autonomously, prints the final application guidance, and transitions to state COMPLETED.

5.3 Message Payload Examples

Eligibility Request Payload

```
{
 "applicant_name": "Fahad Zahid",
 "nationality": "Pakistan",
 "purpose": "study",
 "destination_country": "Finland",
 "university": "University of Jyvaskyla",
 "admission_letter": true
}
```

Eligibility Result Payload

```
{
 "eligible": true,
 "visa_type": "Student Residence Permit",
 "reason": "Applicant is studying in Finland and has a valid admission letter"
}
```

Document Request Payload

```
{
 "passport": true,
 "admission_letter": true,
 "insurance": true,
 "bank_statement": true,
 "available_funds_eur": 5000
}
```

Document Verification Result: Issue Detected

```
{
 "status": "DOCUMENT_ISSUE",
 "missing_documents": [],
 "issue": "Insufficient funds",
 "required_funds_eur": 6720,
 "available_funds_eur": 5000
}
```

Document Verification Result: Accepted

```
{
 "status": "DOCUMENTS_ACCEPTED",
 "missing_documents": [],
 "issue": null,
 "required_funds_eur": 6720,
 "available_funds_eur": 8000
}
```

6. Agent Behaviours

All three agents use CyclicBehaviour. This keeps each agent alive and listening continuously rather than stopping after one message. The agents behave like persistent services, not single-shot scripts.

Agent	Behaviour Class	Key Behaviour
OrchestratorAgent	WorkflowBehaviour	Manages state machine, delegates tasks, aggregates results
EligibilityAgent	EligibilityBehaviour	Listens for profile requests, evaluates eligibility, returns decision
DocumentAgent	DocumentVerificationBehaviour	Listens for document requests, checks completeness and funds, returns result

Inside each behaviour, receive() polls for messages with a short timeout. When a message arrives, the agent parses the JSON body, runs its decision logic, and sends a reply as a new SPADE Message. Because this is non-blocking, all three agents run concurrently in the same Python event loop.

The OrchestratorAgent's WorkflowBehaviour is more involved because it needs to hold state across multiple receive() calls. The state variable tells the behaviour what to do next: send the eligibility request, wait for the result, send the document request, wait for that result, or stop.

7. Test Scenarios and Results

Two scenarios were tested: one where the funds are too low and one where everything passes. Both use the same applicant profile; only the available funds change.

7.1 Scenario A: Insufficient Funds (Primary Demo)

> *This scenario has a student who is eligible but short on funds. The DocumentAgent should catch the shortfall and return a DOCUMENT_ISSUE.*

Applicant Profile

Field	Value
Name	Fahad Zahid
Nationality	Pakistani
Purpose	Study

Field	Value
Destination Country	Finland
University	University of Jyväskylä
Admission Letter	Yes

Document Data

Field	Value
Passport	Present
Admission Letter	Present
Insurance	Present
Bank Statement	Present
Available Funds (EUR)	5,000
Required Funds (EUR)	6,720

Expected System Behaviour

- EligibilityAgent evaluates the profile: purpose is 'study', destination is Finland, admission letter is present. Decision: eligible for Student Residence Permit.
- OrchestratorAgent receives eligibility confirmation and proceeds to document verification.
- DocumentAgent evaluates the document payload: all mandatory documents present, but available funds (EUR 5,000) are below the threshold (EUR 6,720). Decision: DOCUMENT_ISSUE.
- OrchestratorAgent receives the document issue result and outputs the final guidance.

Expected Console Output

```
● (.venv) fahadzahid@Fahads-MacBook-Pro group_95_second_project % cd /Users/fahadzahid/Downloads/age
  nt_tech_/group_95_second_project
  ./run_issue_demo.sh

  [MAIN] Starting Group 95 SPADE immigration MAS prototype...
  [MAIN] Demo scenario selected: issue
  [EligibilityAgent] Started and waiting for eligibility requests.
  [DocumentAgent] Started and waiting for document verification requests.
  [OrchestratorAgent] Started application workflow.
  [OrchestratorAgent] Sending applicant profile to EligibilityAgent...
  [OrchestratorAgent] Applicant profile payload: {"applicant_name": "Fahad Zahid", "nationality": "P
  akistan", "purpose": "study", "destination_country": "Finland", "university": "University of Jyvas
  kyla", "admission_letter": true}
  [EligibilityAgent] Received eligibility request from orchestrator@localhost
  [EligibilityAgent] Decision: eligible for Student Residence Permit.
  [EligibilityAgent] Sent eligibility result to OrchestratorAgent.
  [OrchestratorAgent] Received eligibility result: Student Residence Permit.
  [OrchestratorAgent] Eligibility reason: Applicant is studying in Finland and has a valid admission
   letter.
  [OrchestratorAgent] Eligibility accepted. Sending documents to DocumentAgent...
  [OrchestratorAgent] Uploaded documents payload: {"passport": true, "admission_letter": true, "insu
  rance": true, "bank_statement": true, "available_funds_eur": 5000}
  [DocumentAgent] Received document verification request.
  [DocumentAgent] Decision: insufficient funds detected.
  [DocumentAgent] Sent document verification result to OrchestratorAgent.
  [OrchestratorAgent] Received document verification result.
  [OrchestratorAgent] Action required: Increase financial proof to at least 6720 EUR. Available: 500
  0 EUR.
  [OrchestratorAgent] Final decision: Application not ready for submission.
  [MAIN] Stopping all agents...
```

7.2 Scenario B: Successful Submission (Optional Demo)

> *Same profile, same documents, but funds raised to EUR 8,000. All checks pass and the application comes back ready for submission.*

Modified Document Data

Field	Value
Available Funds (EUR)	8,000
Required Funds (EUR)	6,720
Result	Funds sufficient, threshold met

Expected Console Output

```
(.venv) fahadzahid@Fahads-MacBook-Pro group_95_second_project % cd /Users/fahadzahid/Downloads/age
nt_tech_/group_95_second_project
./run_success_demo.sh
[MAIN] Starting Group 95 SPADE immigration MAS prototype...
[MAIN] Demo scenario selected: success
[EligibilityAgent] Started and waiting for eligibility requests.
[DocumentAgent] Started and waiting for document verification requests.
[OrchestratorAgent] Started application workflow.
[OrchestratorAgent] Sending applicant profile to EligibilityAgent...
[OrchestratorAgent] Applicant profile payload: {"applicant_name": "Fahad Zahid", "nationality": "P
akistan", "purpose": "study", "destination_country": "Finland", "university": "University of Jyvas
kyla", "admission_letter": true}
[EligibilityAgent] Received eligibility request from orchestrator@localhost
[EligibilityAgent] Decision: eligible for Student Residence Permit.
[EligibilityAgent] Sent eligibility result to OrchestratorAgent.
[OrchestratorAgent] Received eligibility result: Student Residence Permit.
[OrchestratorAgent] Eligibility reason: Applicant is studying in Finland and has a valid admission
 letter.
[OrchestratorAgent] Eligibility accepted. Sending documents to DocumentAgent...
[OrchestratorAgent] Uploaded documents payload: {"passport": true, "admission_letter": true, "insu
rance": true, "bank_statement": true, "available_funds_eur": 8000}
[DocumentAgent] Received document verification request.
[DocumentAgent] Decision: documents accepted.
[DocumentAgent] Sent document verification result to OrchestratorAgent.
[OrchestratorAgent] Received document verification result.
[OrchestratorAgent] Final decision: Application ready for submission.
[OrchestratorAgent] Action: Student Residence Permit recommended and documents accepted.
[MAIN] Stopping all agents...
```

8. Technical Setup and Configuration

8.1 Technology Stack

Component	Technology	Purpose
Agent Framework	SPADE 3.x (Python)	Agent lifecycle, behaviours, XMPP messaging

Component	Technology	Purpose
Messaging Protocol	XMPP (Extensible Messaging and Presence Protocol)	Inter-agent message routing
XMPP Server	Prosody (local)	Message broker for agent JIDs
Message Format	JSON over SPADE Message bodies	Structured domain data exchange
Runtime	Python 3.10+	Agent execution environment
Dependency Management	pip + virtualenv	Isolated Python environment

8.2 XMPP Account Configuration

Each agent needs its own XMPP account on Prosody. Credentials are read from environment variables or a .env file. The default setup for local demo uses localhost:

```
ORCHESTRATOR_JID=orchestrator@localhost
ORCHESTRATOR_PASSWORD=password
ELIGIBILITY_JID=eligibility@localhost
ELIGIBILITY_PASSWORD=password
DOCUMENT_JID=document@localhost
DOCUMENT_PASSWORD=password
DEMO_CASE=issue   # or 'success' for Scenario B
```

9. Connection to the Previous GAIA Design

This prototype comes directly from the Group 95 GAIA design. The GAIA phase produced a role model, interaction model, and service model for a seven-agent system. This project implements one focused slice of that.

GAIA Design Element	Implementation
Orchestrator role (central coordinator)	Implemented as OrchestratorAgent with WorkflowBehaviour
Eligibility Assessment role	Implemented as EligibilityAgent with EligibilityBehaviour
Document Verification role	Implemented as DocumentAgent with DocumentVerificationBehaviour
Scenario 1: Student Residence Permit	Selected as the implementation focus for this prototype
XMPP message-based interaction model	Realised using SPADE messaging with performative and ontology fields
Autonomous agent decision-making	Each agent independently evaluates its input and produces a decision
Fee Calculation role	Deferred to future work

GAIA Design Element	Implementation
Appointment Monitoring role	Deferred to future work
Status Tracking role	Deferred to future work

Three agents instead of seven reflects the scope of this phase. The orchestrator, eligibility assessor, and document verifier together cover the core pipeline and are enough to show all required capabilities: multi-agent messaging, domain-specific data exchange, and autonomous decisions.

10. Limitations and Future Work

This prototype was built for the course assignment and has a few clear limitations:

- The eligibility rules are deterministic and hardcoded. A production system would query a rules engine or a database of immigration regulations that can be updated independently.
- The document payload contains boolean flags for document presence. A real system would validate actual uploaded files, check document validity periods, and verify digital signatures.
- No persistent storage is used. Application state exists only in memory during a single run.
- Only one permit category is supported. The full GAIA design covers multiple permit types requiring different eligibility and document rules.
- The XMPP infrastructure uses a local Prosody server. Deployment in a real environment would require a properly secured XMPP service with authentication and transport encryption.

11. Conclusion

The project produces a working SPADE multi-agent system. Three agents (OrchestratorAgent, EligibilityAgent, DocumentAgent) handle a real administrative scenario taken from the GAIA design.

All requirements are met. Agents communicate over XMPP with structured JSON. Each one makes its own decisions: the EligibilityAgent checks profiles against rules, the DocumentAgent checks documents and funds, and the OrchestratorAgent controls the workflow based on what they return.

Both test scenarios work as expected. The insufficient funds case is caught correctly; the successful case clears all checks. The SPADE web GUI lets you inspect agent states, behaviour classes, and message histories while the demo runs.

SPADE works well for this kind of project. The async messaging, CyclicBehaviour, and built-in web GUI made it practical to build, test, and demo a domain-specific MAS inside the time constraints of a course assignment.

12. References

SPADE. (n.d.). ***SPADE documentation*****.** Retrieved April 28, 2026, from https://spade-mas.readthedocs.io

Wooldridge, Michael. (2009). *An introduction to multiagent systems* (2nd ed.). Wiley.

García-Fornes, Ana, Garrido, Ana, Gimeno, José A., Botti, Vicente, & Julián, Vicente. (2020). SPADE 3: Supporting the new generation of multi-agent systems. *IEEE Access, 8*, 182537–182549.

Finnish Immigration Service (Migri). (n.d.). ***Residence permit for studies*****.** Retrieved April 28, 2026, from https://migri.fi/en/studying

University of Jyväskylä. (2026). ***GAIA methodology course materials:*** *Agent Technologies for Developers* [Course materials].